Lecture 22 — 13th Nov

Evaluating wether B is a subsequence of A

$A = a_1 a_2 \dots a_m$

$B = b_1 b_2 \dots b_n$

if $\exists\ i_1 < i_2 < \dots < i_n$ s.t

$a_{i_k} = b_k \quad , \quad k = 1, \dots, n$

then B is a subsequence of A

→ Now evaluating wether B is a circular subsequence of A

look for B in AA

Substring : A subsequence which occur consecutively

→ Naive approach of string matching

Shift by 1 pos. each time
a mismatch occurs

Algorithm Compute_Next (B, m)

Input : B (a string of size m)
Output : next (an array of size m)

begin

next [1] = 1

next [2] = 0

for $i = 3$ to m do

j = next $(i-1) + 1$

while $b_{i-1} \neq b_j$ and $j > 0$ do

j = next $[j] + 1$

next $[i] = j$

end

```
begin
  j = 1 , i = 1 , start = 0;
  while start = 0 and i ≤ n do
      if B[j] = A[i] then
          j = j+1 ; i = i+1 ;
      else
          j = next [j] + 1
          if j = 0 then
              j = 1 , i = i+1 ;
      if j = m+1 then start = i-m,
end
```

1.) for each $v \in V$

$dist[v] \leftarrow \infty$; $parent[v] \leftarrow nil$;

2.) $dist[s] \leftarrow 0$;

3.) Initialise a min - priority queue Q with all vertices of V, keyed by $dist[v]$;

4) while Q is not empty do

$u \leftarrow$ Extract - min (Q)

for each neighbour of u do

if $dis[v] > dist[u] + w[u, v]$ then

$dist[v] \leftarrow dist[u] + w[u, v]$

$parent[v] \leftarrow u$; decrease-key $(Q, v, dist[v])$

return $dist[\,]$, $parent[\,]$